SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I __K.Niharika(192411124)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **K.Niharika**

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drones for Emergency Medicine Delivery

A drone must deliver medicines in a flood-affected area where routes can become blocked and weather changes travel time and risk. It needs a route that is fast but also safe.

i. Greedy Best-First Search: Behaviour, Strengths and Weaknesses

Greedy Best-First Search (GBFS) selects the location that appears closest to the destination using a heuristic, such as straight-line distance. It is often quick, but it does not fully consider the cost already spent or the danger along the route.

GBFS: f(n) = h(n)\nh(n) = estimated distance from the current location to the goal

Strength: quickly focuses on locations that appear closer to the destination.

Weakness: may choose a short-looking route that is flooded, windy, or unsafe.

It does not guarantee the lowest-cost route and can be misled by incomplete information.

ii. How A* Handles the Situation

A* considers both the cost already spent and the estimated remaining cost. For the drone, the cost may include flight time, battery use, and risk.

A*: f(n) = g(n) + h(n)\n

Where : g(n) = cost so far; h(n) = estimated remaining cost

- List the available routes from the drone's current location.
- Calculate g(n), h(n), and f(n) for each candidate.
- Choose the route with the lowest estimated total cost and check its safety.

If a path becomes blocked or weather changes, update the map and replan.

Route	Cost so far	Estimated remaining	Total
A	8 min	4 min	12 min
B	5 min	6 min	11 min
C	3 min	10 min	13 min

In this example, A* selects Route B because its estimated total is 11 minutes, the lowest value.

iii. Heuristic Accuracy and Dynamic Changes

A good heuristic guides the search towards the goal. Straight-line distance is easy to calculate but cannot fully represent storms or blocked areas. An admissible A* heuristic does not overestimate the true remaining cost and can support an optimal solution under standard conditions.

An inaccurate or outdated heuristic can lead either algorithm towards a poor route.

When a path is blocked, the system must remove it from the map and calculate another route.

Frequent updates may require efficient replanning rather than starting from the beginning every time.

iv. Effect of Dynamic Changes and Blocked Paths

A blocked path can make a planned route unusable. Weather can also increase travel time or turn a route unsafe while the drone is in flight. The search system must respond to changes rather than blindly follow the original plan.

- Receive updates from sensors, satellites, authorities, or other drones.
- Remove blocked routes from the graph or mark them as unavailable.
- Update costs when wind, visibility, or terrain conditions change.
- Replan from the current location to avoid repeating the entire search unnecessarily.
- Apply safety rules that reject prohibited or high-risk routes even if they appear fast.

v. Recommendation

A* with real-time replanning is recommended because it considers the route cost and estimated remaining cost. The system should include safety rules, updated weather information, and risk penalties. GBFS may be fast, but A* provides a better balance between route quality and safety.

2. Smart-City Traffic Signal Optimization

The city wants to reduce vehicle waiting time, fuel consumption, and congestion by changing traffic-light timings according to live traffic data.

Problem Formulation

Minimize **J = w1(W) + w2(F) + w3(C)\n**

Where: W = waiting time; F = fuel consumption; C = congestion

The decision variables are green-light durations and signal offsets. Constraints include safe signal changes, pedestrian crossing time, and minimum or maximum green durations.

i. Hill Climbing and Its Limitations

Hill Climbing starts with a signal plan and makes small changes. It keeps a change if the total cost improves, repeating until no nearby change gives a better result.

- Create a starting plan using current traffic counts.
- Try small changes to green-light times.
- Calculate whether each change reduces waiting, fuel use, or congestion.
- Keep an improving plan and repeat; stop when the time limit is reached or no improvement is found.
- It is simple and fast, but it can become trapped in a local optimum and may miss better plans that require several coordinated changes.

ii. Local Optima, Plateaus and Ridges

Problem	Meaning in traffic management
Local optimum	One signal plan looks better than nearby plans, but a better city-wide plan exists.
Plateau	Several timing plans have nearly the same score, so the search makes little progress.
Ridge	Two or more connected signals must change together to improve traffic flow.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA):

Simulated Annealing can sometimes accept a worse solution so it can escape a local optimum. Early in the search it is more willing to accept temporary increases in cost. As its temperature decreases, it becomes more selective.

For a candidate whose cost increases by $\Delta > 0$:\n

Probability of accepting it = **exp(-Δ / T)\n**

T = current temperature, gradually reduced during search

- Can escape local optima by temporarily accepting a worse plan.
- Uses relatively little memory and can be suitable for a single candidate plan.
- Its success depends on the cooling schedule and how neighbouring plans are generated.

Genetic Algorithm (GA):

A Genetic Algorithm maintains a population of candidate signal plans. Each plan is encoded as a chromosome and evaluated using a fitness score. Better plans are more likely to be selected as parents, and crossover and mutation generate new plans for later generations.

- Generate several valid or repairable signal-timing plans.
- Evaluate each plan using the objective function and safety constraints.
- Select stronger plans as parents.
- Use crossover to combine plans and mutation to change selected timings.
- Repair constraint violations and keep the best candidates.
- Repeat for several generations or until the time budget is reached.

A GA explores several parts of the search space at once and may discover coordinated changes across many intersections. However, it can require many evaluations and careful tuning.

iv. Recommendation

A hybrid approach is suitable: use a Genetic Algorithm periodically to find coordinated plans, then use a fast controller to adjust signals using real-time sensor data. Hill Climbing can be used for quick local changes, while safety constraints must always be followed.

3. Autonomous Mars Rover Exploring Unknown Terrain

i. Why Online Search Is Required

An offline search agent assumes the environment is already known. A Mars rover does not have a complete map of rocks, craters, and slopes. It must observe, update its knowledge, act, and then observe again. Therefore, online search is more suitable.

Sense → Update map → Choose safe action → Move → Sense again

ii. Partial Observability and Challenges

- The rover may not see every obstacle because of dust, limited sensors, or hidden terrain. Sensor readings may be inaccurate, and the rover has limited battery and computing power.
- Unknown terrain makes route planning uncertain.
- Rocks, craters, and steep slopes can damage the rover.
- Communication delays mean it may need to make decisions without immediate help.

iii. Building and Updating the Internal Model

- Collect data from cameras and other sensors.
- Estimate the rover's position.
- Mark map areas as free, blocked, unknown, or risky.
- Plan a route to the sample while avoiding hazards.
- Move a short distance, gather new data, and update the map.

iv. Search Strategies and v. Recommendation

Strategy	Description
Uninformed search	Explores without using an estimate of distance to the goal; may examine many unnecessary states.
Informed search (A*)	Uses path cost and a heuristic to find a promising route.
Online search	Uses actions and new observations to update the plan as exploration continues.

The best choice is online planning with an informed route planner such as A*, supported by local obstacle avoidance. Unknown areas should be treated cautiously, and the rover should stop or return to a safe point if sensor information is unreliable. This approach supports adaptability and safety.

v. Recommended Approach: Uncertainty, Adaptability, and Safety

The rover should use online, receding-horizon planning with a heuristic route planner and a safety-aware local controller. It should plan using the current map, move only as far as safety and confidence allow, then replan after receiving new observations.

- Use A* or a similar heuristic planner on the currently known map.
- Use local obstacle avoidance to react to nearby rocks and hazards.
- Assign higher costs to unknown terrain instead of assuming it is safe.
- Use a safe fallback action—stop, reverse, or return to a known waypoint—when sensor confidence is low.
- Prioritize rover survival and mission constraints over the shortest route.

4. University Examination Timetabling as a CSP

A Constraint Satisfaction Problem (CSP) represents a problem using variables, domains, and constraints. The university must assign each exam a time and a hall without creating conflicts.

Component	Example
Variables	Time(E1), Hall(E1), Time(E2), Hall(E2)
Domains	Available time slots and examination halls
Constraints	No student has overlapping exams; halls have enough seats; faculty are available; required subject order is followed.
Solution	A complete timetable that satisfies all hard constraints.

i. Variables, Domains and Constraints

For each exam E_i, define Time(E_i) and Hall(E_i) as variables. If invigilator allocation is included, Invigilator(E_i) can be another variable. The domains list all allowed time slots, halls, and eligible staff.

Important constraints include:

- **Student conflict:** if a student takes both E_i and E_j, their exam times must be different.
- **Hall capacity:** the assigned hall must have enough seats for the number of candidates.
- **Hall availability:** one hall cannot host two exams at the same time.
- **Subject order:** if E_i must occur before E_j, the selected time of E_i must precede E_j.
- **Faculty availability:** an invigilator must be available and cannot supervise overlapping exams.
- **Special accommodations:** accessible halls and approved extra-time arrangements must be provided.

ii. Backtracking Search

Backtracking assigns values one at a time. If a choice breaks a constraint, it returns to the previous decision and tries another value.

- Choose an exam and assign an available time.
- Check student conflicts and subject-order rules.
- Assign a suitable hall if the time is valid.
- Continue with the next exam.
- If no valid choice remains, undo the previous assignment and try another.

Example:

If E1 (Mathematics) and E2 (Programming) share students,\nTime(E1) ≠ Time(E2). If E1 is Monday morning, E2 must use another slot.

iii. Forward Checking and iv. Scalable Strategy

Forward checking removes impossible values from the domains of unassigned exams. For example, after assigning Mathematics on Monday morning, that time is removed from the options for exams taken by the same students. If an exam has no choices left, the system detects the problem early.

- Use the Minimum Remaining Values heuristic to schedule the most restricted exam first.
- Use backtracking and forward checking to reduce unnecessary search.

iv. Real-World Challenges and Scalable Strategy

Large universities may have thousands of students, hundreds of courses, limited halls, and many special requirements. A change to one exam can affect several others. Student enrolment may change, or a hall may become unavailable at short notice.

- Represent conflicts efficiently, for example with a graph whose nodes are exams and whose edges connect exams sharing students.
- Use backtracking with MRV, Degree heuristic, and forward checking for initial timetable construction.
- Separate hard constraints from soft preferences and minimize penalties for soft-constraint violations.
- Use local repair to fix a valid timetable when small changes occur.
- Allow administrators to lock confirmed exams so later updates disturb as few assignments as possible.

A practical solution is a CSP solver with backtracking, MRV/degree heuristics, and forward checking, followed by local optimization to improve preferences. This is more efficient than brute force while preserving hard constraints.

5. Strategic Game AI for Real-Time Decision Making

i. Minimax Algorithm

Minimax is used in competitive turn-based games. The AI (MAX) tries to get the highest score, while the opponent (MIN) tries to reduce that score. The algorithm examines possible moves and opponent responses, then works backward to select the best move.

MAX chooses the highest child value; MIN chooses the lowest child value.

AI move	Possible response scores	Worst-case value
A	3, 5	min(3,5) = 3
B	2, 9	min(2,9) = 2
C	4, 6	min(4,6) = 4

The AI chooses Move C because its worst-case score, 4, is higher than the worst-case scores of A and B.

ii. Computational Challenges

If each state has b possible moves and the search looks ahead d levels, the number of nodes can grow approximately as $O(b^d)$. Even a moderate number of moves per state becomes expensive as the depth increases.

- Many legal moves may be available at every state.
- Each opponent response creates more branches to examine.
- Deeper search improves foresight but increases computation exponentially.
- The AI must finish before the real-time deadline.
- If search ends too early, the evaluation of a non-terminal state may be inaccurate.

The AI must therefore prioritize promising branches, reuse earlier calculations, and stop safely before the deadline while keeping the best move found so far.

iii. Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by skipping branches that cannot change the final decision. Alpha is the best value MAX can guarantee so far; beta is the best value MIN can guarantee.

- If $\alpha \geq \beta$, the remaining branches at that point can be pruned.
- Evaluate a possible move and pass alpha and beta values to its children.
- Update alpha at MAX nodes and beta at MIN nodes.
- When alpha becomes greater than or equal to beta, stop checking the remaining branches there.
- Continue until the best move is identified.

Alpha-Beta pruning gives the same decision as Minimax for the same search tree, but often examines fewer nodes. Checking promising moves first makes pruning more effective.

iv. Evaluation Function

When the search does not reach a final win or loss, an evaluation function estimates how good a game state is for the AI.

Score = 5M + 3E + 4R + 2P + 6O − 5D\n

M = material/resources; E = economy; R = map control;\nP = objective progress; O = objective safety; D = danger from opponent

The factors include the number and strength of units, available resources, control of important areas, progress towards objectives, and the safety of the base. The weights are examples and should be tested and adjusted for the game.

v. Recommendation for Real-Time Performance

Use Minimax with Alpha-Beta pruning and iterative deepening. Iterative deepening searches increasing depths and keeps the best move from the deepest completed search if time runs out.

- Order promising moves first to improve pruning.
- Reuse previously evaluated states when possible.
- Keep the evaluation function fast.
- Use a strict time limit so the AI always responds on time.

This approach balances decision quality and speed for turn-based games. Very large or simultaneous-action games may require additional heuristics or other search methods.

Conclusion

The best AI method depends on the information available, the size of the search space, safety requirements, and time limits.

Drone delivery: A* with real-time replanning helps balance cost and safety.

Traffic control: a Genetic Algorithm combined with a fast controller can optimize and adapt signal timings.

Mars rover: online search updates the plan as new terrain is discovered.

Exam timetabling: CSP with backtracking and forward checking reduces conflicts.

Game AI: Minimax with Alpha-Beta pruning improves decisions while reducing computation.

Overall, effective AI systems combine suitable algorithms with accurate information, safety checks, and the ability to update decisions when conditions change.